

DOORS

Módulo de administración de procesos y eventos

Obligatorio

Programación II
Universidad de Montevideo

Erika Puhl
Mikaela Scotti

Índice

Objetivo.....	2
Requerimientos.....	2
Requerimientos funcionales.....	2
Requerimientos no funcionales.....	4
Supuestos y decisiones tomadas.....	5
Diseño del modelo y Diagrama UML.....	7
Estructuras de datos utilizadas.....	10
Análisis de complejidad.....	12
pload.....	12
pprepate.....	13
pexecute.....	15
pfinish.....	17
pstatus.....	18
DoorsLogger.....	20
ProcessConsole.....	21
Manejo de errores y excepciones.....	21
Pruebas realizadas.....	23
Conclusión.....	31

Objetivo

El objetivo del proyecto es diseñar e implementar un módulo de administración de procesos para Doors, un sistema operativo desarrollado por la empresa MacrosUM. Este módulo se encarga de gestionar los procesos de distintos usuarios, controlando su carga, preparación, ejecución, finalización y consulta de estados.

El sistema permite organizar el ciclo de vida de cada proceso desde que es cargado, mediante archivos .csv, hasta que finaliza. Para esto, cada proceso almacena información como su PID, nombre, usuario propietario, estado, prioridad y eventos asociados. A partir de estos datos, el módulo calcula la prioridad de los procesos, mediante una formula, y determina el orden en que deben ejecutarse.

Además, el módulo registra los cambios importantes que ocurren sobre los procesos, como el pasaje a pendiente, el inicio de ejecución, la finalización y el desborde de la pila de procesos finalizados. Esto permite mantener un seguimiento de las acciones realizadas dentro del sistema.

El propósito principal es resolver la gestión del ciclo de vida de los procesos dentro del sistema Doors, utilizando estructuras de datos para organizar la información de forma eficiente y cumplir con los requerimientos del sistema.

Requerimientos

Requerimientos funcionales

El proyecto debe permitir cargar usuarios y procesos desde archivos .csv mediante el comando pload. Los procesos cargados ingresan con estado NEW y quedan almacenados en una cola de nuevos procesos, sin prioridad calculada.

Cada proceso debe registrarse con un PID único, nombre, usuario propietario, estado, prioridad y eventos asociados. Los estados posibles son NEW, PENDING, RUNNING y FINISHED.

Cada proceso está compuesto por eventos. Un evento representa una operación asociada al uso de un recurso del sistema, pudiendo ser de tipo CPU, RAM o DISK. Los eventos CPU representan operaciones de procesamiento, los eventos RAM representan operaciones

sobre memoria principal y los eventos DISK representan operaciones sobre almacenamiento en disco. Esta clasificación se utiliza tanto para registrar la ejecución del proceso como para calcular su prioridad.

Mediante el comando pprepare, se deben tomar los procesos almacenados en la cola de nuevos procesos, calcular su prioridad ante el resto de los procesos y moverlos a la estructura de procesos pendientes. En esta operación, el estado del proceso cambia de NEW a PENDING.

La prioridad se calcula para determinar el orden de ejecución de los procesos pendientes. Como el sistema operativo Doors ejecuta un único proceso a la vez, es necesario definir cuál será el próximo proceso en pasar a ejecución. Para eso, los procesos pendientes deben mantenerse ordenados por su prioridad, quedando primero aquel con mayor valor de $P(p)$.

La fórmula utilizada para calcular la prioridad es:

$$P(p) = \left(\frac{8N_{CPU}(p.events) + 2N_{RAM}(p.events) + 2N_{DISK}(p.events)}{N_{events}(p.events)} \right) + W(p.user) \cdot N_{events}(p.events)$$
$$W(u) = \begin{cases} 32, & \text{si } u = \text{ADMIN} \\ 16, & \text{si } u = \text{GENERIC} \end{cases}$$

Donde:

- $P(p)$: prioridad del proceso p .
- $N_{CPU}(p.events)$: cantidad de eventos de tipo CPU del proceso.
- $N_{RAM}(p.events)$: cantidad de eventos de tipo RAM del proceso.
- $N_{DISK}(p.events)$: cantidad de eventos de tipo DISK del proceso.
- $N_{events}(p.events)$: cantidad total de eventos del proceso.
- $W(p.user)$: peso asociado al tipo de usuario propietario del proceso.

Cada usuario se identifica mediante un UID numérico único, un alias y un tipo. Los tipos posibles son ADMIN y GENERIC. El peso $W(p.user)$ es un coeficiente usado para modificar la prioridad del proceso según el tipo de usuario. Para usuarios ADMIN, el peso es 32, para usuarios GENERIC, el peso es 16. Por eso, ante procesos con eventos similares, el proceso de un usuario ADMIN tendrá mayor prioridad que el de un usuario GENERIC.

Al pasar un proceso de NEW a PENDING, debe registrarse ese cambio en el log, incluyendo los datos principales del proceso, el usuario propietario y la prioridad calculada.

El comando `pexecute` debe ejecutar el proceso pendiente con mayor prioridad. Al comenzar la ejecución, el proceso pasa a estado `RUNNING`. Dado que Doors es monotarea, no puede existir más de un proceso en ejecución simultáneamente.

Durante la ejecución, debe registrarse en el log la información del proceso ejecutado y el detalle de sus eventos asociados.

El comando `pfinish` debe finalizar el proceso actualmente en ejecución. La finalización puede ser `OK`, `ERROR` o `TERMINATED`. En el caso de `TERMINATED`, se debe indicar el UID del usuario que fuerza la finalización.

Una vez finalizado, el proceso debe almacenarse en una pila de procesos finalizados. Esta pila tiene una capacidad máxima definida por una constante. Cuando la pila alcanza su capacidad máxima y se debe agregar un nuevo proceso finalizado, debe registrarse en el log el contenido de la pila en orden inverso al de finalización.

El comando `pstatus` debe permitir consultar el estado actual de los procesos cargados en memoria. La consulta general muestra el proceso en ejecución, los procesos pendientes y los últimos procesos finalizados almacenados.

También deben permitirse consultas específicas: `pstatus -verbose` para mostrar el detalle completo de los eventos, `pstatus -u [UID]` para filtrar por usuario y `pstatus -p [PID]` para consultar un proceso específico con todos sus eventos asociados.

Finalmente, debe generarse un archivo de log con los eventos principales del ciclo de vida de los procesos: preparación, ejecución, finalización y desborde de la pila de finalizados.

Requerimientos no funcionales

La solución debe implementarse en Java, utilizando el proyecto base proporcionado para el obligatorio.

Para la implementación deben utilizarse únicamente los TADs provistos en la librería del proyecto. No se permite usar TADs de terceros ni estructuras de datos provistas por Java u otras librerías externas. Cada estructura elegida debe justificarse según las operaciones que resuelve y el orden de tiempo esperado.

Las funciones principales deben incluir análisis de complejidad temporal mediante notación Big O. Además, las consultas no pueden resolverse utilizando estructuras intermedias que

almacenen resultados previamente calculados. Los resultados deben obtenerse en el momento en que se ejecuta cada consulta.

Doors debe respetar su condición de sistema monotarea. Por lo tanto, no puede existir más de un proceso en estado RUNNING al mismo tiempo. Las consultas realizadas con `pstatus` deben considerar únicamente los procesos cargados en memoria al momento de ejecutar el comando. Los procesos descartados de la pila de finalizados no deben ser considerados en consultas posteriores.

La pila de procesos finalizados debe respetar una capacidad máxima definida mediante una constante del sistema. Cuando se alcance esa capacidad, el comportamiento debe ajustarse a lo definido para el manejo del desborde de la pila.

El archivo de log debe generarse con el nombre `DOORS_PROCESS_LOG_FECHA`, utilizando el formato `yyyy-MM-dd` para la fecha. Los timestamps registrados dentro del archivo deben utilizar el formato `yyyy-MM-dd HH:mm:ss`.

Supuestos y decisiones tomadas

Se decidió separar la escritura del log en una clase auxiliar llamada `DoorsLogger`. Esta clase centraliza la creación y escritura del archivo de log, permitiendo que `ProcessManagerImpl` se encargue solamente de la lógica de administración de procesos.

Para generar el archivo se utiliza `Path`, construyendo el nombre `DOORS_PROCESS_LOG_FECHA` con la fecha actual en formato `yyyy-MM-dd`. Para escribir los registros se utiliza `BufferedWriter` junto con `Files.newBufferedWriter`, de forma que el archivo se crea si no existe y, si ya existe, los nuevos registros se agregan al final sin borrar el contenido anterior.

Cada entrada del log incluye un timestamp con el formato `yyyy-MM-dd HH:mm:ss`. Además, la clase permite registrar tanto líneas simples como bloques de texto, lo cual se utiliza para guardar eventos como la preparación de procesos, la ejecución, la finalización y el desborde de la pila de procesos finalizados.

También se decidió representar el proceso en ejecución mediante una única variable de tipo `Proceso`, llamada `procesoEnEjecucion`, y no mediante una estructura de datos. Esto se debe a que Doors es un sistema monotarea, por lo tanto en ningún momento puede haber más de un proceso en estado RUNNING.

Otra decisión fue mantener la capacidad máxima de procesos finalizados como una constante en la interfaz ProcessManager, llamada MAX_FINISHED_PROCESS_ON_RAM. Este valor representa una regla fija del sistema y permite controlar cuántos procesos finalizados pueden permanecer en memoria antes de registrar el desborde en el log.

También se decidió mantener separados los estados generales del proceso y los estados de finalización. Para esto se utilizan las enumeraciones EstadoProceso y EstadoFinalizacion. EstadoProceso indica en qué etapa del ciclo de vida se encuentra el proceso, mientras que EstadoFinalizacion indica cómo terminó cuando ya pasó a estado FINISHED.

Se asumió que primero deben cargarse los usuarios y luego los procesos. Esto se debe a que cada proceso tiene asociado un usuario propietario, por lo tanto al leer el archivo de procesos es necesario que los usuarios ya estén cargados en memoria para poder validar que el UID exista.

Se decidió utilizar el PID como identificador único de cada proceso y el UID como identificador único de cada usuario. Gracias a esto, el sistema puede evitar duplicados y realizar búsquedas directas mediante las estructuras hash utilizadas en ProcessManagerImpl.

Otra decisión fue que el peso asociado al tipo de usuario se mantenga dentro de la clase Usuario, mediante el método getPeso(). De esta forma, la clase Proceso no necesita conocer directamente cuánto vale un usuario ADMIN o GENERIC, sino que obtiene ese valor desde el usuario propietario al calcular la prioridad.

También se asumió que el comando pfinish TERM UID representa una finalización forzada realizada por un usuario del sistema. Por eso, el UID recibido en el comando se utiliza para identificar al usuario responsable de la terminación, aunque no necesariamente sea el usuario propietario del proceso.

Por último, se decidió separar la interacción por consola de la lógica principal del sistema. La clase ProcessConsole se encarga de leer e interpretar los comandos ingresados por el usuario, mientras que ProcessManagerImpl ejecuta las operaciones correspondientes. Esto permite que la consola no administre directamente los procesos, sino que actúe como intermediaria entre el usuario y el sistema.

Diseño del modelo y Diagrama UML

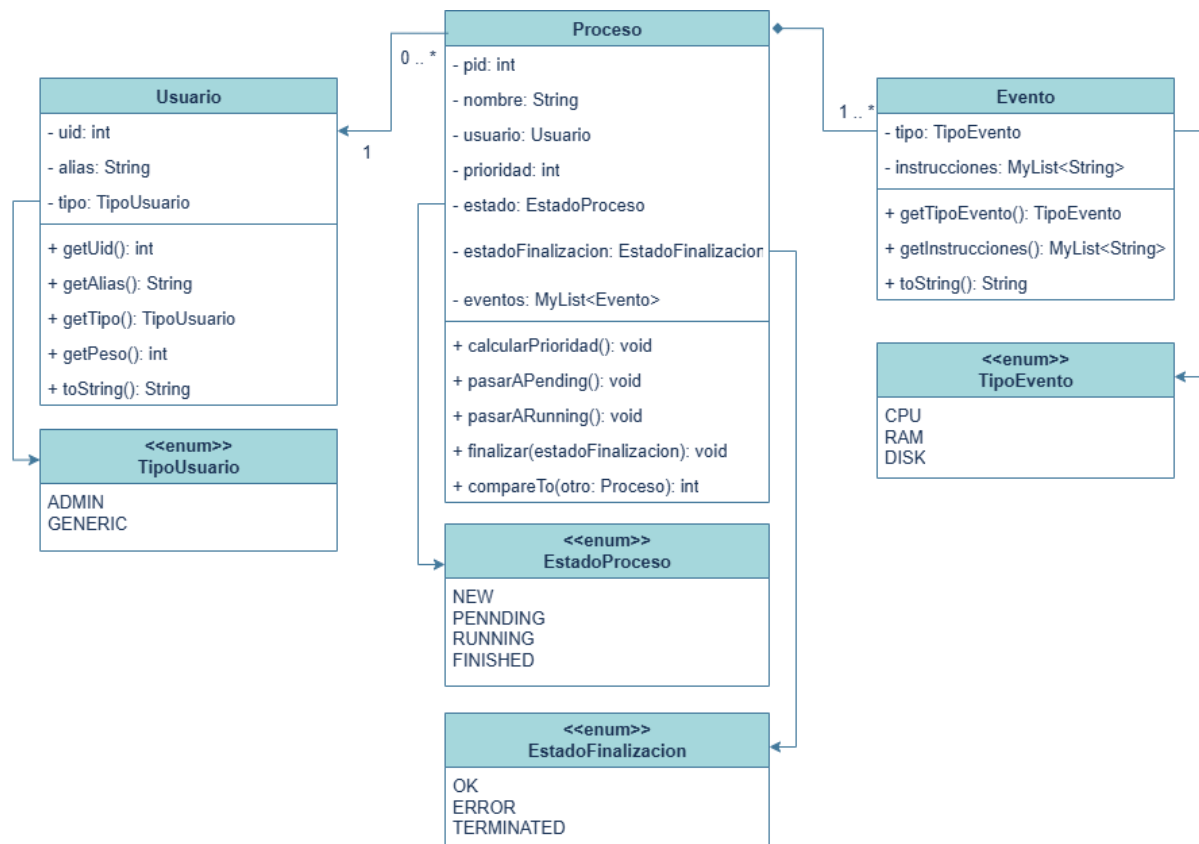


Figura 1: Diagrama UML del modelo de dominio

A partir de los requisitos del sistema se identificaron tres clases principales del dominio: Usuario, Proceso y Evento. Además, se definieron cuatro enumeraciones para representar valores fijos del sistema: TipoUsuario, TipoEvento, EstadoProceso y EstadoFinalizacion.

La clase Usuario representa a los usuarios que pueden tener procesos asociados dentro del sistema. Cada usuario se identifica mediante un uid, un alias y un tipo. El uid y el alias se definieron como atributos finales, ya que son datos que identifican al usuario y no deberían modificarse luego de su creación. El atributo tipo se representa mediante la enumeración TipoUsuario, cuyos valores posibles son ADMIN y GENERIC.

Esta clasificación influye en el cálculo de prioridad de los procesos. Para eso, la clase Usuario cuenta con el método getPeso(), que devuelve el valor correspondiente según el tipo de usuario: 32 para usuarios ADMIN y 16 para usuarios GENERIC. Además, la clase tiene métodos de acceso como getUid(), getAlias() y getTipo(), para obtener el id, alias y el tipo de usuario.

La relación entre Usuario y Proceso se modela como una asociación. Un usuario puede tener cero o varios procesos asociados, pero cada proceso pertenece a un único usuario. Por eso, la clase Proceso contiene un atributo de tipo Usuario.

La clase Proceso representa un proceso administrado por el sistema. Cada proceso tiene un pid, un nombre, un usuario propietario, una prioridad, un estado, un estadoFinalizacion y una lista de eventos. Al crearse, el proceso ingresa con estado NEW, con una prioridad inicial en 0 y sin estado de finalización, ya que todavía no fue ejecutado ni finalizado.

El estado general del proceso se representa mediante la enumeración EstadoProceso, cuyos valores son NEW, PENDING, RUNNING y FINISHED. Estos valores permiten representar el ciclo de vida del proceso dentro del sistema. Por otra parte, el estado de finalización se representa mediante la enumeración EstadoFinalizacion, cuyos valores son OK, ERROR y TERMINATED. Este último no representa una etapa del ciclo de vida, sino la forma en que terminó el proceso cuando ya llegó a FINISHED.

La clase Proceso incluye métodos para modificar su estado y resolver operaciones propias del modelo. El método calcularPrioridad() calcula la prioridad usando los eventos del proceso y el peso del usuario propietario. Los métodos pasarAPending() y pasarARunning() permiten cambiar el estado del proceso según avance en el sistema. El método finalizar() cambia el proceso a estado FINISHED y guarda su estado de finalización.

Además, Proceso implementa Comparable<Proceso>. Esto permite comparar procesos según su prioridad, lo cual es necesario para almacenarlos en una estructura de pendientes donde el próximo proceso a ejecutar debe ser el de mayor prioridad.

La clase Evento representa una acción que forma parte de la ejecución de un proceso. Cada evento tiene un tipo y una lista de instrucciones. El tipo se representa mediante la enumeración TipoEvento, cuyos valores posibles son CPU, RAM y DISK. Esta clasificación permite distinguir qué recurso del sistema utiliza cada evento y también se usa para calcular la prioridad del proceso.

Las instrucciones del evento se almacenan en una lista MyList<String>, ya que un evento puede tener una o más instrucciones y esa cantidad puede variar según la información cargada desde el archivo CSV. La clase cuenta con métodos como getTipoEvento(), getInstrucciones() y toString(), utilizados para consultar y mostrar la información del evento.

La relación entre Proceso y Evento se modela como una composición. Esto se debe a que un proceso está compuesto por uno o más eventos, y esos eventos no se consideran

entidades independientes dentro del sistema. Cada evento pertenece a un único proceso, y cada proceso debe tener asociado al menos un evento.

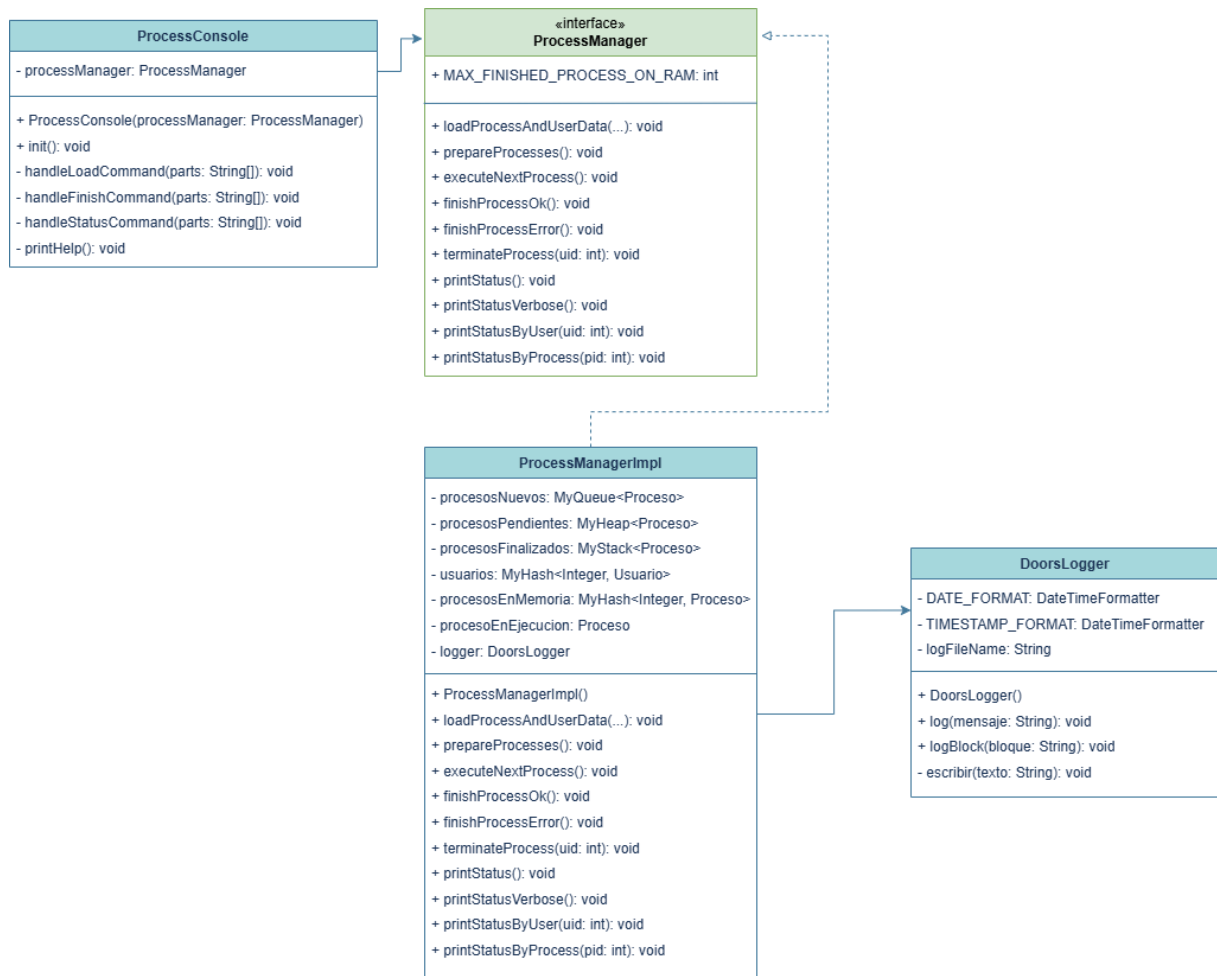


Figura 2: Diagrama UML de administración y consola del sistema

Además de las clases del dominio, el sistema utiliza algunas clases auxiliares para separar responsabilidades. La interfaz `ProcessManager` define las operaciones principales del módulo, como cargar usuarios y procesos, preparar procesos, ejecutar el próximo proceso pendiente, finalizar el proceso en ejecución y consultar el estado del sistema. Esta interfaz funciona como contrato entre la consola y la implementación real.

La clase `ProcessManagerImpl` es la implementación principal de esa interfaz. En ella se concentra la lógica de administración de procesos. Esta clase mantiene las estructuras necesarias para representar los procesos nuevos, pendientes, en ejecución y finalizados, además de los usuarios y procesos cargados actualmente en memoria. También se encarga de realizar la carga desde archivos CSV, calcular prioridades, cambiar estados, ejecutar procesos, finalizarlos y responder las consultas de estado.

La clase `ProcessConsole` se utiliza como punto de entrada para la interacción por consola. Esta clase no administra directamente los procesos, sino que lee los comandos ingresados por el usuario, interpreta sus parámetros, realiza validaciones básicas y llama al método correspondiente de `ProcessManager`. De esta forma, la entrada por consola queda separada de la lógica principal del sistema.

Por último, se incorporó la clase auxiliar `DoorsLogger`, encargada de centralizar la escritura del archivo de log. Esta clase genera el nombre del archivo según la fecha actual, agrega un timestamp a cada registro y permite guardar los eventos principales del sistema, como la preparación, ejecución, finalización y desborde de la pila de procesos finalizados.

Estructuras de datos utilizadas

Para la implementación del sistema se utilizaron distintas estructuras según la responsabilidad de cada clase. La idea principal fue separar la entrada por consola, la administración de procesos, la escritura del log y el almacenamiento de la información en memoria.

En la clase `ProcessConsole` no se utilizan estructuras de datos complejas, ya que su función no es almacenar procesos ni usuarios, sino recibir comandos desde la consola. Para esto se utiliza `Scanner`, que permite leer lo que ingresa el usuario. Además, para el comando `pload` se utilizan `Path` y `Files` para trabajar con las rutas recibidas y verificar que correspondan a archivos existentes y legibles antes de llamar al administrador de procesos. Esta clase solo interpreta comandos como `pload`, `pprepare`, `pexecute`, `pfinish` y `pstatus`, y luego delega la operación en `ProcessManager`.

La interfaz `ProcessManager` define las operaciones principales que debe cumplir el módulo. En esta interfaz también se define la constante `MAX_FINISHED_PROCESS_ON_RAM`, que indica la capacidad máxima de la pila de procesos finalizados. Se decidió definir este valor como constante porque representa una regla fija del sistema y debe poder ser utilizada por la implementación principal.

La clase `ProcessManagerImpl` concentra la lógica principal del sistema. En esta clase se mantienen en memoria los usuarios, los procesos y las estructuras que representan el ciclo de vida de cada proceso. Cada estructura fue elegida según el estado del proceso y las operaciones que se necesitan realizar.

Para la lectura de los archivos CSV se utiliza Path junto con BufferedReader. Path permite representar la ubicación del archivo recibido por consola, mientras que BufferedReader permite leer el contenido línea por línea. Esto se utiliza en la carga de usuarios y procesos, donde cada línea del archivo se interpreta y se transforma en objetos del sistema.

Para los procesos nuevos se utiliza una cola, representada por MyQueue<Proceso> procesosNuevos. Esta estructura se eligió porque los procesos cargados con pload ingresan inicialmente con estado NEW. Como todavía no fueron preparados, quedan esperando hasta que se ejecute pprepare. La cola permite mantener el orden de llegada y retirar los procesos uno por uno para calcular su prioridad y pasarlos a pendientes.

Para los procesos pendientes se utiliza un heap, representado por MyHeap<Proceso> procesosPendientes. Esta estructura se eligió porque los procesos en estado PENDING deben ejecutarse según su prioridad. Como el próximo proceso a ejecutar debe ser el de mayor prioridad, el heap permite insertar procesos pendientes y obtener de forma eficiente el proceso con mayor prioridad cuando se ejecuta pexecute.

Para los procesos finalizados se utiliza una pila, representada por MyStack<Proceso> procesosFinalizados. Esta estructura se eligió porque el sistema debe conservar los últimos procesos finalizados en memoria. Al ser una pila, permite acceder primero al proceso finalizado más reciente. Además, cuando se alcanza la capacidad máxima definida por MAX_FINISHED_PROCESS_ON_RAM, la pila se vacía en orden inverso al de finalización y ese contenido se registra en el log.

Para almacenar los usuarios cargados se utiliza un hash, representado por MyHash<Integer, Usuario> usuarios. La clave utilizada es el UID del usuario. Esta estructura permite buscar rápidamente un usuario específico, lo cual es necesario al cargar procesos desde el CSV, al verificar que el usuario propietario exista y al validar el UID responsable en una finalización forzada con pfinish TERM [UID].

También se utiliza un hash para los procesos cargados en memoria, representado por MyHash<Integer, Proceso> procesosEnMemoria. En este caso, la clave utilizada es el PID del proceso. Esta estructura permite consultar directamente un proceso mediante pstatus -p [PID], controlar que no se carguen procesos duplicados y eliminar de memoria los procesos descartados cuando ocurre un desborde de la pila de finalizados.

El proceso en ejecución se representa mediante una única variable de tipo Proceso, llamada procesoEnEjecucion. No se utiliza una cola ni otra estructura para esto porque Doors es un sistema monotarea. Esto significa que solo puede existir un proceso en estado RUNNING al

mismo tiempo. Si esta variable contiene un proceso, el sistema no permite ejecutar otro hasta que el actual finalice.

En la clase Proceso se utiliza `MyList<Evento>` para almacenar los eventos asociados al proceso. Esta estructura permite guardar una cantidad variable de eventos, ya que cada proceso puede tener uno o más eventos cargados desde el archivo CSV. También se utiliza para recorrer los eventos al calcular la prioridad del proceso.

En la clase Evento se utiliza `MyList<String>` para almacenar las instrucciones del evento. Esta estructura se eligió porque cada evento puede tener una o más instrucciones, y la cantidad no es fija. De esta forma, cada evento puede guardar todas las instrucciones que fueron cargadas desde el CSV.

Por último, la clase DoorsLogger no utiliza un TAD del proyecto, ya que su responsabilidad no es administrar procesos, sino escribir en un archivo. Para esto se utiliza `Path`, que representa la ubicación del archivo de log, y `BufferedWriter`, que permite escribir texto en el archivo de forma ordenada. El archivo se crea si no existe y, si ya existe, los nuevos registros se agregan al final sin borrar el contenido anterior. Se decidió separar esta lógica en una clase propia para no mezclar la administración de procesos con la escritura del log.

Análisis de complejidad

pload

```
@Override 1 usage
public void loadProcessAndUserData(String processCsvPath, String usersCsvPath) {

    cargarUsuarios(usersCsvPath);
    cargarProcesos(processCsvPath);

    System.out.println("Carga finalizada.");
    System.out.println("Usuarios cargados: " + usuarios.size());
    System.out.println("Procesos cargados en NEW: " + procesosNuevos.size());
}
```

Figura 3: Implementación del método `loadProcessAndUserData` (Comando `pload`).

El primer algoritmo analizado es el correspondiente al comando `pload`, encargado de cargar los usuarios y procesos desde los archivos CSV. Este algoritmo se encuentra principalmente en la clase `ProcessManagerImpl`, dentro del método `loadProcessAndUserData()`. También intervienen los métodos auxiliares `cargarUsuarios()`, `cargarProcesos()`, `parsearEventos()` y `parsearInstrucciones()`.

El comando pload primero carga los usuarios y luego los procesos. Este orden es necesario porque cada proceso tiene asociado un usuario propietario. Por lo tanto, antes de crear un proceso, el sistema debe poder verificar que el UID del usuario exista en la estructura de usuarios cargados.

La carga de usuarios recorre el archivo de usuarios línea por línea. Por cada línea se obtienen los datos del usuario, se valida que el UID no esté repetido y luego se inserta el usuario en el hash usuarios.

Luego se carga el archivo de procesos. Por cada proceso se valida que el PID no esté repetido, se busca el usuario propietario en el hash de usuarios y se interpretan los eventos e instrucciones asociados. Si el proceso es válido, se guarda en la cola de procesos nuevos y también en el hash de procesos en memoria.

Para simplificar el análisis, se define n como la cantidad total de datos procesados durante la carga, incluyendo usuarios, procesos, eventos e instrucciones leídas desde los archivos CSV. Como cada dato se recorre y procesa una vez, la complejidad temporal del comando pload es $O(n)$.

Esto significa que el tiempo de ejecución crece de forma lineal con el tamaño de los archivos de entrada. Si los archivos tienen más usuarios, procesos, eventos o instrucciones, el tiempo aumenta en proporción a esa cantidad de información cargada.

ppprepare

```
public void prepareProcesses() {
    while (!procesosNuevos.isEmpty()) {
        try {Proceso proceso = procesosNuevos.dequeue();

            proceso.calcularPrioridad();
            proceso.setEstado(EstadoProceso.PENDING);

            procesosPendientes.insert(proceso);

            String mensaje = "NEW PENDING PROCESS: PID=" + proceso.getPid()
                + " | " + proceso.getNombre()
                + " | USER:" + proceso.getUsuario().getAlias()
                + " UID:" + proceso.getUsuario().getUid()
                + " | P=" + proceso.getPrioridad();

            System.out.println(mensaje);
            logger.log(mensaje);

        } catch (EmptyQueueException e) {
            System.out.println("No hay procesos nuevos para preparar.");
        } catch (EstadoProcesoInvalidoException e) {
            System.out.println("No se pudo cambiar el proceso a estado PENDING.");
        }
    }
}
```

Figura 4: Implementación del método prepareProcesses (Comando pprepare).

El comando pprepare toma los procesos que se encuentran en la cola de procesos nuevos, es decir, aquellos que tienen estado NEW. Mientras la cola no esté vacía, se retira un proceso, se calcula su prioridad, se cambia su estado a PENDING y se inserta en el heap de procesos pendientes.

El cálculo de prioridad recorre los eventos del proceso para contar cuántos son de tipo CPU, RAM y DISK. Luego se aplica la fórmula de prioridad definida por la consigna, usando también el peso del usuario propietario. Por eso, el costo de calcular la prioridad depende de la cantidad de eventos que tenga cada proceso.

Si p es la cantidad de procesos nuevos que se preparan y E es la cantidad total de eventos de esos procesos, el cálculo de prioridades tiene costo $O(E)$. Además, cada proceso se inserta en el heap de pendientes, y cada inserción en el heap tiene costo $O(\log p)$. Como se insertan p procesos, esa parte tiene costo $O(p \log p)$.

Por lo tanto, la complejidad queda como $O(E + p \log p)$.

Esto significa que el tiempo depende de la cantidad de eventos que deben recorrerse para calcular prioridades y de la cantidad de procesos que deben insertarse en el heap de pendientes.

pexecute

```
@Override Usage
public void executeNextProcess() {
    if (procesoEnEjecucion != null) {
        System.out.println("Ya existe un proceso en ejecución.");
        return;
    }

    if (procesosPendientes.isEmpty()) {
        System.out.println("No hay procesos pendientes para ejecutar.");
        return;
    }

    try {
        Proceso proceso = procesosPendientes.remove();

        proceso.setEstado(EstadoProceso.RUNNING);
        procesoEnEjecucion = proceso;

        String bloque = "EXECUTING PROCESS: PID=" + proceso.getPid()
            + " | USER:" + proceso.getUsuario().getAlias()
            + " UID:" + proceso.getUsuario().getUid();

        System.out.println(bloque);

        MyList<Evento> eventos = proceso.getEventos();

        for (int i = 0; i < eventos.size(); i++) {
            Evento evento = eventos.get(i);

            String lineaEvento = "EVENT: " + evento.getTipoEvento()
                + " | Instructions " + instruccionesToString(evento.getInstrucciones());

            System.out.println(lineaEvento);
            bloque += System.lineSeparator() + lineaEvento;
        }

        logger.logBlock(bloque);
    } catch (EmptyHeapException e) {
        System.out.println("No hay procesos pendientes para ejecutar.");
    } catch (EstadoProcesoInvalidoException e) {
        System.out.println("No se pudo cambiar el proceso a estado RUNNING.");
    }
}
```

Figura 5: Implementación del método executeNextProcess (Comando pexecute).

El comando pexecute se encarga de ejecutar el próximo proceso pendiente. Si no hay un proceso ejecutándose, el sistema verifica si existen procesos pendientes en el heap procesosPendientes. En caso de que el heap no esté vacío, se extrae el proceso con mayor prioridad. Esta es la razón por la que se utiliza un heap para los procesos pendientes: permite obtener el proceso prioritario sin recorrer todos los procesos cada vez que se ejecuta pexecute.

Luego de obtener el proceso, se cambia su estado a RUNNING, se guarda en la variable procesoEnEjecucion y se registra su ejecución. Además, se muestran y registran los eventos asociados al proceso, junto con sus instrucciones.

Si p es la cantidad de procesos pendientes, extraer el proceso con mayor prioridad desde el heap tiene costo $O(\log p)$. Las validaciones iniciales y el cambio de estado tienen costo $O(1)$.

Por otra parte, al ejecutar el proceso también se recorren sus eventos e instrucciones para mostrarlos por consola y registrarlos en el log. Si e es la cantidad de eventos del proceso ejecutado e i la cantidad total de instrucciones de esos eventos, esta parte tendría un costo $O(e + i)$.

Por lo tanto, considerando también la impresión y registro de eventos, la complejidad temporal de pexecute es de $O(\log p + e + i)$

Si solo se analiza la selección del próximo proceso a ejecutar, sin contar la salida por consola ni el log, la complejidad principal es $O(\log p)$, debido a la extracción del heap.pfinish

pfinish

```
private void finalizarProceso(EstadoFinalizacion estadoFinalizacion, Integer uidResponsable) {
    if (procesoEnEjecucion == null) {
        System.out.println("No hay proceso en ejecución.");
        return;
    }

    Usuario usuarioResponsable = null;

    if (estadoFinalizacion == EstadoFinalizacion.TERMINATED) {
        usuarioResponsable = usuarios.get(uidResponsable);

        if (usuarioResponsable == null) {
            System.out.println("No existe el usuario responsable de la terminación.");
            return;
        }
    }

    Proceso proceso = procesoEnEjecucion;
    try {
        proceso.setEstado(EstadoProceso.FINISHED);
        proceso.setEstadoFinalizacion(estadoFinalizacion);
    } catch (EstadoProcesoInvalidoException e) {
        System.out.println("No se pudo cambiar el proceso a estado FINISHED.");
        return;
    } catch (EstadoFinalizacionInvalidoException e) {
        System.out.println("El estado de finalización no es válido.");
        return;
    }

    String mensaje;

    if (estadoFinalizacion == EstadoFinalizacion.TERMINATED) {
        mensaje = "ENDING PROCESS: PID=" + proceso.getPid()
            + " | STATE: TERMINATED by USER:" + usuarioResponsable.getAlias()
            + " UID:" + usuarioResponsable.getUid();
    } else {
        mensaje = "ENDING PROCESS: PID=" + proceso.getPid()
            + " | STATE: " + estadoFinalizacion;
    }

    System.out.println(mensaje);
    logger.log(mensaje);

    agregarProcesoFinalizado(proceso);
    procesoEnEjecucion = null;
}
```

Figura 6: Implementación de la finalización de procesos (Comando pfinish).

El cuarto algoritmo analizado es el correspondiente al comando pfinish. Este comando permite finalizar el proceso que se encuentra en ejecución. El algoritmo se encuentra en la clase ProcessManagerImpl, principalmente en los métodos finishProcessOk(), finishProcessError(), terminateProcess() y finalizarProceso().

El comando puede ejecutarse de tres formas: pfinish OK, pfinish ERROR o pfinish TERM [UID]. En los dos primeros casos, el sistema finaliza el proceso en ejecución con estado de finalización OK o ERROR. En el caso de TERM, además se debe validar que el UID recibido

corresponda a un usuario existente, ya que ese usuario queda registrado como responsable de la finalización forzada.

Primero se verifica que exista un proceso actualmente en ejecución. Si no hay ningún proceso en estado RUNNING, el sistema informa el error y no realiza la finalización. Esta validación tiene costo $O(1)$, ya que el proceso en ejecución se guarda en una única variable llamada `procesoEnEjecucion`.

Luego se cambia el estado del proceso a FINISHED, se asigna su estado de finalización y se registra la finalización en el log. Después, el proceso se agrega a la pila de procesos finalizados mediante el método `agregarProcesoFinalizado()`.

En el caso normal, agregar un proceso a la pila tiene costo $O(1)$, ya que solo se inserta el proceso finalizado en `procesosFinalizados`. Sin embargo, si la pila alcanza la capacidad máxima definida por `MAX_FINISHED_PROCESS_ON_RAM`, se produce un desborde. En ese caso, el sistema recorre la pila para registrar en el log los procesos finalizados y luego los elimina de memoria. Si la capacidad máxima de la pila se considera una constante, esta operación sigue siendo $O(1)$. Si se analiza de forma general, siendo f la cantidad de procesos finalizados almacenados en la pila, el desborde tiene costo $O(f)$.

Para `pfinish TERM [UID]`, además se busca el usuario responsable en el hash de usuarios. Esta búsqueda tiene costo promedio $O(1)$, ya que se utiliza el UID como clave.

Por lo tanto, en el caso normal, la complejidad temporal de `pfinish` es $O(1)$. En el caso de desborde de la pila, la complejidad puede expresarse como $O(f)$, donde f es la cantidad de procesos finalizados que deben recorrerse para registrarlos y eliminarlos de memoria.

Como en este sistema la capacidad máxima de la pila de finalizados es una constante, el comando `pfinish` puede considerarse $O(1)$ para la implementación realizada.

`pstatus`

```
@Override 1 usage
public void printStatus() {
    System.out.println("PROCESS STATUS");

    System.out.println("EXECUTING:");
    if (procesoEnEjecucion == null) {
        System.out.println("No hay proceso en ejecución.");
    } else {
        imprimirProcesoResumen(procesoEnEjecucion);
    }

    System.out.println("PENDING:");
    imprimirPendientesOrdenados(verbose: false);

    System.out.println("FINISHED:");
    imprimirFinalizadosDesdePila(verbose: false);
}
```

Figura 7: Implementación de las consultas de estado (Comando pstatus).

Las consultas de estado del sistema se resuelven mediante los métodos, `printStatus()`, `printStatusVerbose()`, `printStatusByUser()` y `printStatusByProcess()` dentro de `ProcessManagerImpl`.

El comando `psstatus` muestra el estado general del sistema. Primero consulta el proceso en ejecución, lo cual tiene costo $O(1)$ porque se guarda en una única variable llamada `procesoEnEjecucion`. Luego muestra los procesos pendientes. Para esto se recorren los procesos cargados en memoria y se insertan los pendientes en un heap auxiliar, con el objetivo de mostrarlos ordenados por prioridad sin modificar el heap original. Por esta razón, en el peor caso la complejidad de esta parte es $O(p \log p)$, donde p es la cantidad de procesos cargados en memoria.

La variante `psstatus -verbose` funciona de forma similar, pero además muestra los eventos e instrucciones de los procesos. Por eso, a la complejidad anterior se le suma el recorrido de esa información interna. Si E representa la cantidad de eventos mostrados e I la cantidad de instrucciones mostradas, su complejidad es $O(p \log p + E + I)$.

En el caso de `psstatus -u [UID]`, primero se busca el usuario en el hash de usuarios, lo cual tiene costo promedio $O(1)$. Luego se recorren los procesos cargados en memoria para encontrar los que pertenecen a ese usuario. Como no se guarda una estructura separada por usuario, esta búsqueda tiene costo $O(p)$. Si además se imprimen eventos e instrucciones, la complejidad queda $O(p + E + I)$.

Por último, `psstatus -p [PID]` es más directo, porque busca el proceso en el hash `procesosEnMemoria` usando el PID como clave. Esta búsqueda tiene costo promedio $O(1)$. Luego, si se muestran sus eventos e instrucciones, el costo depende de la cantidad de

información del proceso consultado. Por lo tanto, su complejidad es $O(e + i)$, donde e es la cantidad de eventos del proceso e i la cantidad de instrucciones asociadas.

DoorsLogger

```
public void log(String mensaje) { 2 usages  @mikascotti10 *
    String timestamp = LocalDateTime.now().format(TIMESTAMP_FORMAT);
    String linea = "[" + timestamp + "]: " + mensaje;
    escribir(linea);
}

public void logBlock(String bloque) { 2 usages  @mikascotti10 *
    String timestamp = LocalDateTime.now().format(TIMESTAMP_FORMAT);
    String texto = "[" + timestamp + "]: " + bloque;
    escribir(texto);
}

private void escribir(String texto) { 2 usages  @mikascotti10 +1 *
    try (BufferedWriter writer = Files.newBufferedWriter(logFilePath,
        StandardOpenOption.CREATE, StandardOpenOption.APPEND)) {
        writer.write(texto);
        writer.newLine();
    } catch (IOException e) {
        System.out.println("No se pudo escribir en el archivo de log." + e.getMessage());
    }
}
```

Figura 8: Implementación de la escritura de registros en la clase DoorsLogger

El algoritmo corresponde a la escritura del archivo de log implementada en la clase DoorsLogger. Esta clase se utiliza para registrar eventos importantes del sistema, como la preparación de procesos, la ejecución, la finalización y el desborde de la pila de procesos finalizados.

Los métodos principales son `log()`, `logBlock()` y `escribir()`. Los métodos `log()` y `logBlock()` agregan un timestamp al mensaje recibido y luego llaman al método `escribir()`, que abre el archivo de log y agrega el registro al final del archivo.

Para escribir el archivo se utiliza `BufferedWriter` junto con `Files.newBufferedWriter`. Si el archivo no existe, se crea. si ya existe, el nuevo contenido se agrega sin borrar los registros anteriores.

La complejidad temporal depende del tamaño del texto que se escriba. Si m representa la cantidad de caracteres del mensaje o bloque registrado, la escritura tiene complejidad $O(m)$, ya que el sistema debe recorrer y escribir ese contenido en el archivo.

En el caso de un registro simple, el tamaño del mensaje suele ser pequeño, por lo que puede considerarse cercano a $O(1)$. Sin embargo, cuando se registra un bloque con varios

eventos o varios procesos finalizados, la complejidad crece según la cantidad de texto generado.

ProcessConsole

La lectura y validación de los comandos por consola está implementada en la clase `ProcessConsole`. Esta clase no administra directamente los procesos, sino que recibe el comando ingresado por el usuario, valida su formato y llama al método correspondiente de `ProcessManager`.

El método principal es `init()`, que mantiene la consola activa mientras el usuario no ingrese `exit` o `quit`. En cada iteración se lee una línea, se separa el comando en partes y se identifica qué operación debe ejecutarse.

Las validaciones realizadas en la consola tienen costo bajo, ya que los comandos tienen una cantidad fija y reducida de parámetros. Por ejemplo, `pload` valida las opciones `-p` y `-u`, `pfinish` valida si la finalización es `OK`, `ERROR` o `TERM [UID]`, y `pstatus` valida sus variantes disponibles.

Por lo tanto, la complejidad de procesar un comando en `ProcessConsole` puede considerarse $O(1)$, ya que no depende de la cantidad de procesos, usuarios o eventos cargados en el sistema. El costo importante aparece después, cuando la consola delega la operación en `ProcessManagerImpl`, donde se ejecutan los algoritmos principales ya analizados.

Manejo de errores y excepciones

Para controlar los errores del sistema se crearon excepciones propias dentro del paquete `exceptions`. Estas excepciones permiten diferenciar los distintos tipos de datos inválidos y evitar que las clases principales del modelo se creen con información incorrecta.

En las clases del dominio se realizan validaciones al momento de crear o modificar objetos. En la clase `Usuario` no se permite crear un usuario con `UID` negativo ni con alias nulo o vacío. También se valida el tipo de usuario, por lo que si el valor recibido no es `ADMIN` o `GENERIC`, se lanza una excepción de tipo `TipoUsuarioInvalidoException`.

La excepción `DatoInvalidoException` se utiliza para validar datos generales del sistema, como identificadores numéricos, nombres, alias o referencias obligatorias. Por ejemplo, se

utiliza cuando se intenta crear un proceso con PID inválido, un usuario con alias vacío o un proceso sin usuario asociado.

La excepción `TipoUsuarioInvalidoException` se utiliza cuando el tipo de usuario recibido no es un valor definido por el sistema, que son ADMIN y GENERIC.

La excepción `TipoEventoInvalidoException` se utiliza cuando el tipo de evento recibido no son los valores permitidos. Los únicos tipos aceptados son CPU, RAM y DISK.

La excepción `InstruccionesInvalidasException` se utiliza para controlar que cada evento tenga al menos una instrucción asociada. No se permite crear un evento con una lista de instrucciones nula, vacía o con instrucciones sin contenido.

La excepción `EventosInvalidosException` se utiliza en la clase `Proceso` para validar que todo proceso tenga uno o más eventos asociados. Esto evita que se carguen procesos sin eventos.

También se utilizan `EstadoProcesoInvalidoException` y `EstadoFinalizacionInvalidoException` para validar los estados del proceso y su forma de finalización. Esto asegura que los estados pertenezcan a las enumeraciones definidas por el sistema y evita asignar valores no contemplados.

Además de las excepciones del modelo, la clase `ProcessConsole` contempla errores asociados al uso incorrecto de comandos. Cuando el usuario ingresa un comando no reconocido, se muestra un mensaje indicando que el comando es inválido y se sugiere utilizar help.

Para el comando `pload`, se valida que se ingresen correctamente las opciones `-p` y `-u`, correspondientes al archivo de procesos y al archivo de usuarios. Además, antes de delegar la carga al administrador de procesos, se verifica que las rutas existan, que sean archivos regulares y que puedan leerse.

En el caso de `pfinish`, se valida que el usuario indique un tipo de finalización válido: OK, ERROR o TERM [UID]. Si se utiliza TERM, también se controla que el UID recibido sea numérico.

Para `pstatus`, se validan las variantes disponibles: `-verbose`, `-u [UID]` y `-p [PID]`. En las consultas por usuario o por proceso, se controla que el UID o PID ingresado sea un número válido antes de llamar a la operación correspondiente.

En `ProcessManagerImpl` se contemplan distintos casos de error para mantener la consistencia del sistema. Durante la carga de usuarios y procesos desde archivos CSV, se ignoran líneas con formato inválido, usuarios duplicados, procesos duplicados y procesos asociados a un UID inexistente. Esto permite continuar con la carga de los datos válidos sin detener todo el sistema por una línea incorrecta.

También se controla que no se pueda ejecutar un proceso si ya existe otro en estado `RUNNING`, respetando la condición monotarea de Doors. Si no hay procesos pendientes, el comando `pexecute` informa que no existen procesos disponibles para ejecutar.

En la finalización de procesos se valida que exista un proceso actualmente en ejecución. Para el caso de `pfinish TERM UID`, también se verifica que el UID recibido corresponda a un usuario cargado en memoria. Si el usuario no existe, la finalización no se realiza.

Cuando la pila de procesos finalizados alcanza su capacidad máxima, se registra el desborde en el log, se muestran los procesos almacenados en orden inverso al de finalización y se eliminan de la estructura de procesos cargados en memoria para que no sean considerados en consultas posteriores.

Por último, si ocurre un error al escribir en el archivo de log, el sistema captura la excepción de entrada/salida e informa por consola que no se pudo escribir el registro. De esta forma, un problema en la escritura del log no detiene completamente la ejecución del sistema.

Pruebas realizadas

Se realizaron pruebas con JUnit para comprobar que las clases principales del sistema funcionen correctamente. La idea fue probar primero las clases del dominio, después el flujo principal del administrador de procesos y, por último, los TADs que se usan como base para manejar la información.

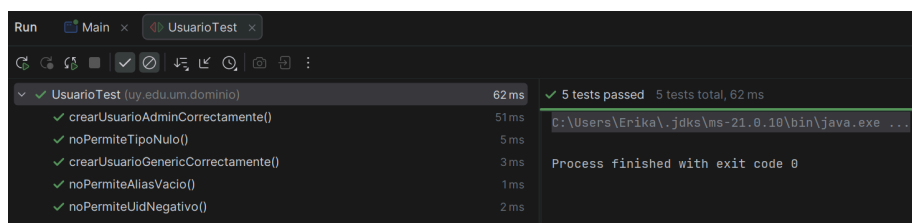


Figura 9: Test JUnit UsuarioTest

En primer lugar, se probaron las clases del dominio. En `UsuarioTest` se verificó que los usuarios se creen correctamente, que se asigne bien el tipo de usuario y que el peso sea el

esperado según sea ADMIN o GENERIC. También se probaron casos inválidos, como UID negativo, alias vacío o tipo de usuario nulo.

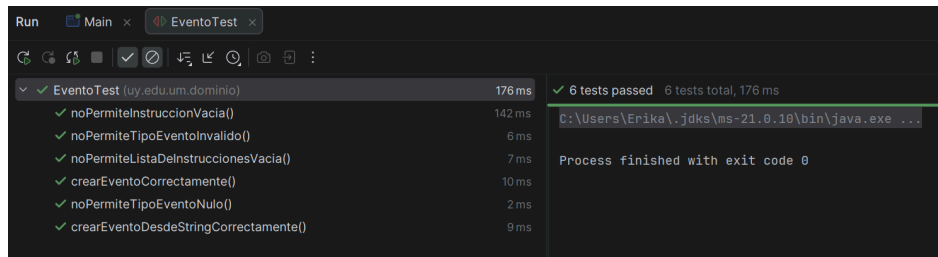


Figura 10: Test JUnit de EventoTest

Luego, en EventoTest, se comprobó que los eventos se creen correctamente con sus tipos e instrucciones. También se probaron casos donde el evento no debería crearse, por ejemplo si el tipo no es válido o si no tiene instrucciones asociadas.

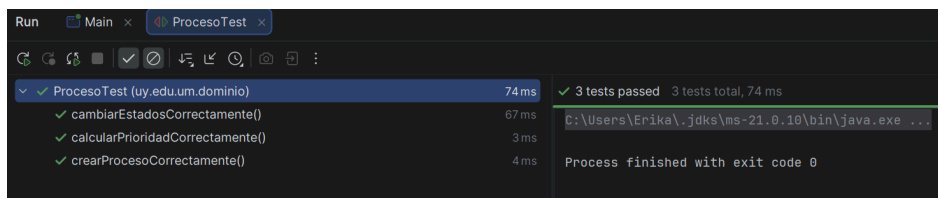


Figura 11: Test JUnit de ProcesoTest

En ProcesoTest se probó la creación de procesos, el cálculo de prioridad y los cambios de estado. Esto permitió verificar que un proceso pueda pasar por los estados NEW, PENDING, RUNNING y FINISHED, y que al finalizar guarde correctamente su estado de finalización.

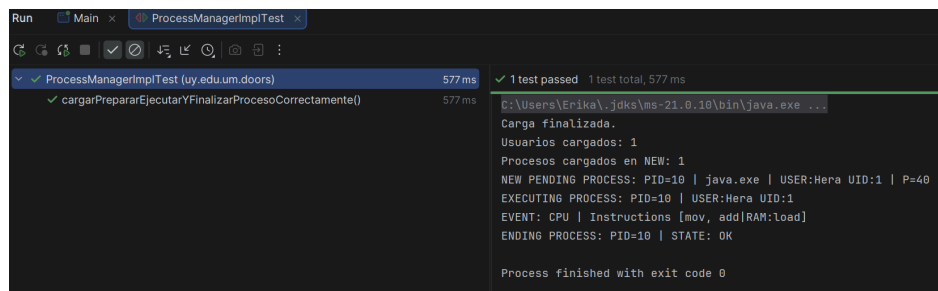


Figura 12: Test JUnit de ProcessManager

Después se probó ProcessManagerImpl, que es la clase que une la mayor parte del funcionamiento del sistema. En esta prueba se cargaron usuarios y procesos desde archivos CSV, se prepararon los procesos, se ejecutó el siguiente proceso pendiente y se finalizó

correctamente. Esta prueba sirve para comprobar que el flujo principal funciona de manera integrada, no solo por clases separadas.

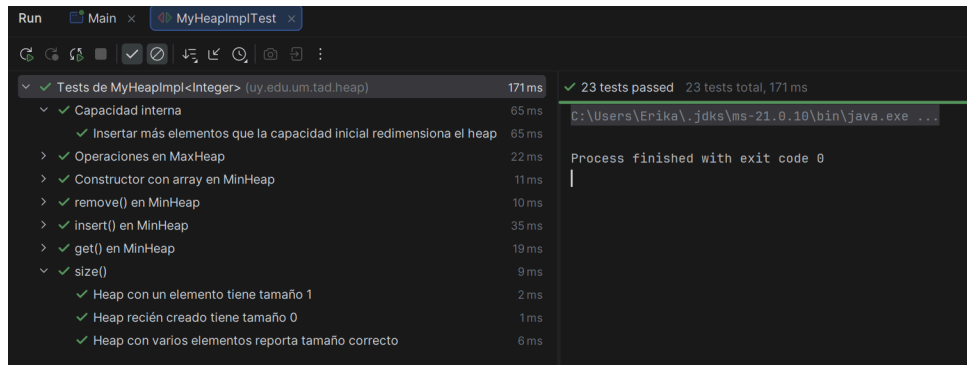


Figura 13: Test JUnit de MyHeapImplTest

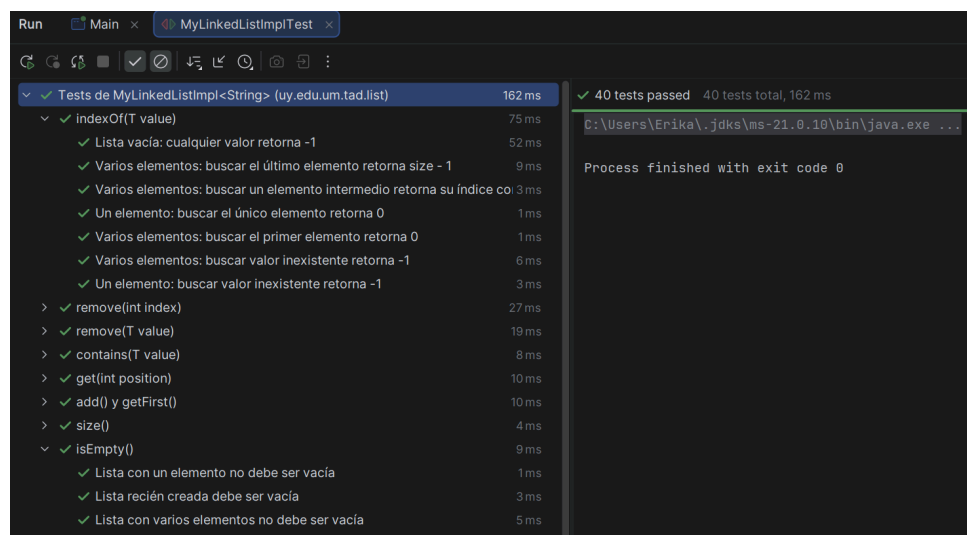


Figura 14: Test JUnit de MyLinkedListImplTest

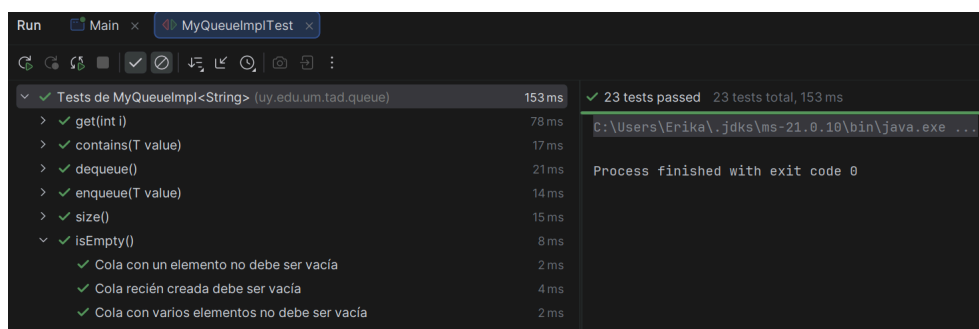


Figura 15: Test JUnit de MyQueueImplTest

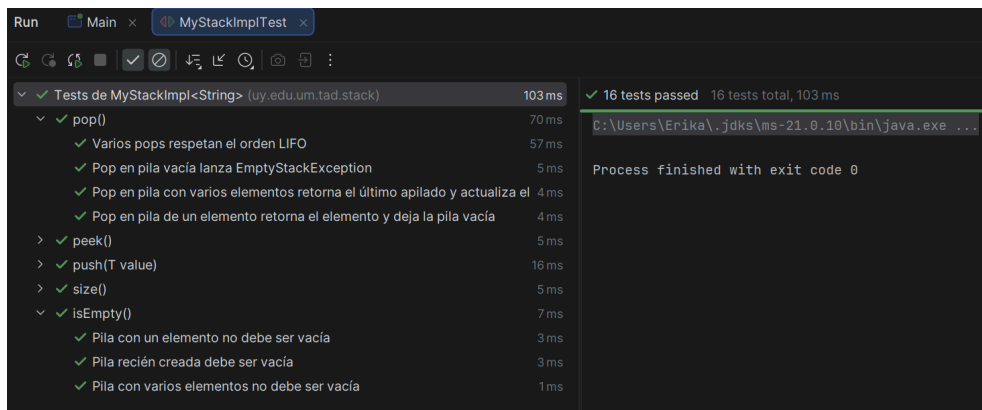


Figura 16: Test JUnit de MyStackImplTest

Además, se ejecutaron pruebas JUnit sobre los TADs utilizados en el proyecto. Se probaron la lista, la cola, la pila y el heap, mediante los tests de MyLinkedListImpl, MyQueueImpl, MyStackImpl y MyHeapImpl. Esto es importante porque estas estructuras son las que después utiliza ProcessManagerImpl para guardar procesos nuevos, procesos pendientes, procesos finalizados y otros datos del sistema.

Luego de las pruebas con JUnit, también se realizaron pruebas manuales desde la consola para verificar el funcionamiento completo de los comandos del sistema.

```
doors> pload -p process.csv -u users.csv
Carga finalizada.
Usuarios cargados: 12
Procesos cargados en NEW: 500
```

Figura 17: Prueba manual del comando pload

En esta prueba se ejecutó el comando `pload -p process.csv -u users.csv`, utilizando los archivos CSV de procesos y usuarios. Como resultado, el sistema cargó correctamente 12 usuarios y 500 procesos en estado NEW.

```
doors> pprepare
NEW PENDING PROCESS: PID=51331 | powershell.exe | USER:Artemis UID:4086 | P=83
NEW PENDING PROCESS: PID=47122 | services.exe | USER:Hermes UID:642 | P=85
NEW PENDING PROCESS: PID=57160 | powershell.exe | USER:Apollo UID:731 | P=115
NEW PENDING PROCESS: PID=19520 | taskmgr.exe | USER:Zeus UID:25 | P=131
NEW PENDING PROCESS: PID=69402 | netstat.exe | USER:Ares UID:98 | P=117
NEW PENDING PROCESS: PID=54724 | gradle.exe | USER:Zeus UID:25 | P=163
NEW PENDING PROCESS: PID=55995 | ftp.exe | USER:Prometheus UID:1024 | P=133
NEW PENDING PROCESS: PID=11014 | cmd.exe | USER:Hades UID:219 | P=84
NEW PENDING PROCESS: PID=83656 | ssh.exe | USER:Artemis UID:4086 | P=52
NEW PENDING PROCESS: PID=24752 | ssh.exe | USER:Zeus UID:25 | P=166
NEW PENDING PROCESS: PID=36887 | docker.exe | USER:Hades UID:219 | P=50
```

Figura 18: Prueba manual del comando pprepare

Luego se ejecutó el comando `pprepare`. Como era la primera vez que se preparaban los procesos, el sistema tomó los 500 procesos cargados en estado NEW, calculó su prioridad y los pasó a estado PENDING. En la consola se puede observar que cada proceso queda registrado como NEW PENDING PROCESS, mostrando su PID, nombre, usuario propietario, UID y prioridad calculada.

```
doors> pexecute
EXECUTING PROCESS: PID=58753 | USER:Hera UID:87
EVENT: RAM | Instructions [load, move, read, memcpy]
EVENT: CPU | Instructions [mov, inc, sub, jump, mod, pop, jmp, sub]
EVENT: CPU | Instructions [push, pop, sub, xor, loop]
EVENT: CPU | Instructions [push, div, mod, test, ret, div, jump]
EVENT: RAM | Instructions [calloc, fetch, malloc, copy, alloc, alloc, write]
EVENT: CPU | Instructions [call, mul, test, mov, ret, call, div, cmp]
EVENT: RAM | Instructions [cache, alloc, move, copy, store, memcpy]
```

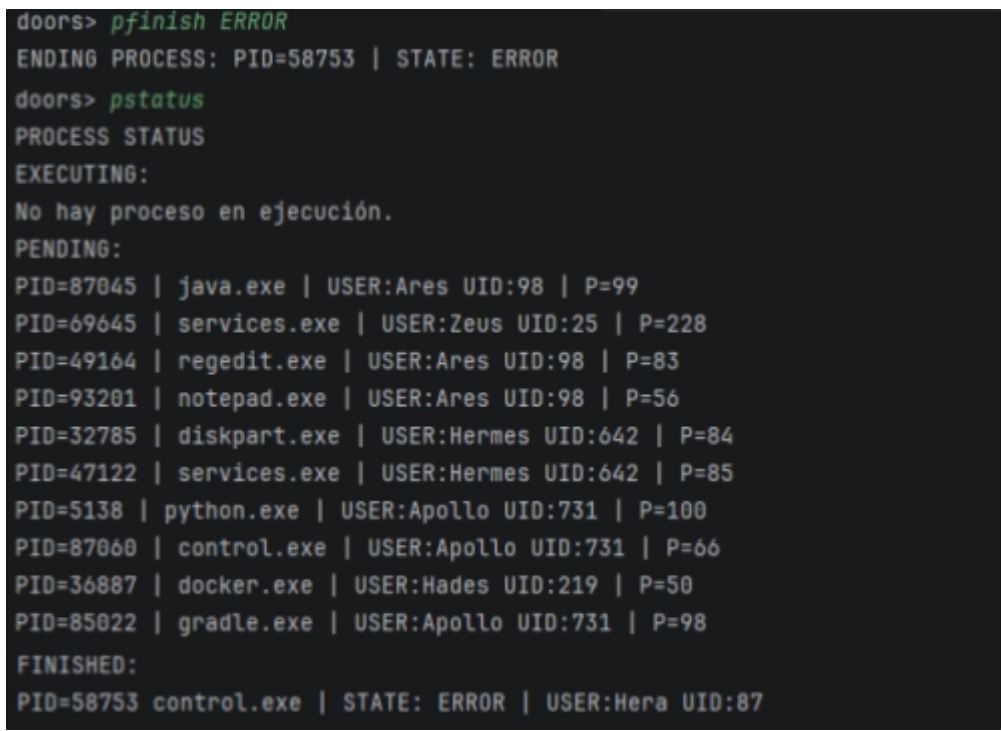
Figura 19: Prueba manual del comando `pexecute`

Después se ejecutó el comando `pexecute`. El sistema tomó el proceso pendiente con mayor prioridad, lo cambió a estado RUNNING y mostró sus eventos asociados. En esta prueba se observa que el proceso ejecutado fue PID=58753, perteneciente al usuario Hera con UID 87. También se muestran sus eventos de tipo RAM y CPU, junto con las instrucciones correspondientes.

```
doors> pstatus
PROCESS STATUS
EXECUTING:
PID=58753 | control.exe | USER:Hera UID:87 | P=229
PENDING:
PID=87045 | java.exe | USER:Ares UID:98 | P=99
PID=69645 | services.exe | USER:Zeus UID:25 | P=228
PID=49164 | regedit.exe | USER:Ares UID:98 | P=83
PID=93201 | notepad.exe | USER:Ares UID:98 | P=56
PID=32785 | diskpart.exe | USER:Hermes UID:642 | P=84
PID=47122 | services.exe | USER:Hermes UID:642 | P=85
PID=5138 | python.exe | USER:Apollo UID:731 | P=100
PID=87060 | control.exe | USER:Apollo UID:731 | P=66
PID=36887 | docker.exe | USER:Hades UID:219 | P=50
PID=85022 | gradle.exe | USER:Apollo UID:731 | P=98
PID=72684 | wmic.exe | USER:Hera UID:87 | P=163
PID=64496 | control.exe | USER:Cronos UID:333 | P=116
PID=38897 | cmd.exe | USER:Artemis UID:4086 | P=116
PID=50169 | taskmgr.exe | USER:Demeter UID:1507 | P=70
PID=17404 | control.exe | USER:Hera UID:87 | P=100
FINISHED:
No hay procesos finalizados en memoria.
```

Figura 20: Prueba manual del comando pstatus con un proceso en ejecución

Luego se ejecutó pstatus para observar el estado general del sistema. En ese momento había un proceso en ejecución, 499 procesos pendientes y ningún proceso finalizado en memoria. Esto confirma que, luego de ejecutar un proceso, el sistema lo quita de la estructura de pendientes y lo mantiene como único proceso en estado RUNNING.



```
doors> pfinish ERROR
ENDING PROCESS: PID=58753 | STATE: ERROR
doors> pstatus
PROCESS STATUS
EXECUTING:
No hay proceso en ejecución.
PENDING:
PID=87045 | java.exe | USER:Ares UID:98 | P=99
PID=69645 | services.exe | USER:Zeus UID:25 | P=228
PID=49164 | regedit.exe | USER:Ares UID:98 | P=83
PID=93201 | notepad.exe | USER:Ares UID:98 | P=56
PID=32785 | diskpart.exe | USER:Hermes UID:642 | P=84
PID=47122 | services.exe | USER:Hermes UID:642 | P=85
PID=5138 | python.exe | USER:Apollo UID:731 | P=100
PID=87060 | control.exe | USER:Apollo UID:731 | P=66
PID=36887 | docker.exe | USER:Hades UID:219 | P=50
PID=85022 | gradle.exe | USER:Apollo UID:731 | P=98
FINISHED:
PID=58753 control.exe | STATE: ERROR | USER:Hera UID:87
```

Figura 21: Prueba manual de finalización con pfinish ERROR

En esta prueba se ejecutó pfinish ERROR sobre el proceso que estaba en ejecución. El sistema finalizó el proceso PID=58753 con estado de finalización ERROR. Luego, al consultar nuevamente con pstatus, se observa que ya no hay proceso en ejecución y que el proceso finalizado aparece en la pila de finalizados con estado ERROR.

```
doors> pfinish TERM 25
ENDING PROCESS: PID=48693 | STATE: TERMINATED by USER:Zeus UID:25
doors> pstatus
PROCESS STATUS
EXECUTING:
No hay proceso en ejecución.
PENDING:
PID=87045 | java.exe | USER:Ares UID:98 | P=99
PID=69645 | services.exe | USER:Zeus UID:25 | P=228
PID=49164 | regedit.exe | USER:Ares UID:98 | P=83
PID=93201 | notepad.exe | USER:Ares UID:98 | P=56
PID=32785 | diskpart.exe | USER:Hermes UID:642 | P=84
PID=47122 | services.exe | USER:Hermes UID:642 | P=85
PID=64496 | control.exe | USER:Cronos UID:333 | P=116
PID=38897 | cmd.exe | USER:Artemis UID:4086 | P=116
PID=50169 | taskmgr.exe | USER:Demeter UID:1507 | P=70
PID=17404 | control.exe | USER:Hera UID:87 | P=100
FINISHED:
PID=58753 control.exe | STATE: ERROR | USER:Hera UID:87
PID=48693 node.exe | STATE: TERMINATED | USER:Prometheus UID:1024
```

Figura 22: Prueba manual de finalización forzada con pfinish TERM UID

Después se ejecutó nuevamente pexecute para iniciar otro proceso y luego se probó la finalización forzada con pfinish TERM 25. En este caso, el UID 25 el cual es el id del usuario Zeus. El sistema finalizó el proceso en ejecución con estado TERMINATED y registró al usuario Zeus como responsable de la terminación.

```

doors> pexecute
EXECUTING PROCESS: PID=50801 | USER:Prometheus UID:1024
EVENT: CPU | Instructions [loop, jump]
EVENT: CPU | Instructions [div, sub, ret, sub, mov]
EVENT: CPU | Instructions [mul, mod, mod, jump, add]
EVENT: CPU | Instructions [inc, dec, inc, loop, jmp]
EVENT: RAM | Instructions [store, fetch, malloc, copy]
EVENT: RAM | Instructions [store, calloc, calloc, store]
EVENT: RAM | Instructions [write, alloc, load, fetch]
doors> pfinish OK
ENDING PROCESS: PID=50801 | STATE: OK
doors> pstatus
PROCESS STATUS
EXECUTING:
No hay proceso en ejecución.
PENDING:
PID=87045 | java.exe | USER:Ares UID:98 | P=99
PID=69645 | services.exe | USER:Zeus UID:25 | P=228
PID=49164 | regedit.exe | USER:Ares UID:98 | P=83
PID=93201 | notepad.exe | USER:Ares UID:98 | P=56
PID=32785 | diskpart.exe | USER:Hermes UID:642 | P=84
PID=47122 | services.exe | USER:Hermes UID:642 | P=85
PID=5138 | python.exe | USER:Apollo UID:731 | P=100
PID=87060 | control.exe | USER:Apollo UID:731 | P=66
PID=36887 | docker.exe | USER:Hades UID:219 | P=50
PID=85022 | gradle.exe | USER:Apollo UID:731 | P=98
PID=16417 | ssh.exe | USER:Hermes UID:642 | P=99
FINISHED:
PID=58753 control.exe | STATE: ERROR | USER:Hera UID:87
PID=48693 node.exe | STATE: TERMINATED | USER:Prometheus UID:1024
PID=50801 control.exe | STATE: OK | USER:Prometheus UID:1024

```

Figura 23: Prueba manual de finalización correcta con pfinish OK

Luego se volvió a ejecutar un proceso y se probó la finalización correcta mediante pfinish OK. El proceso ejecutado fue finalizado con estado OK y agregado a la pila de procesos finalizados. Con esto se verificaron las tres formas de finalización posibles: ERROR, TERMINATED y OK.

```

doors> pexecute
EXECUTING PROCESS: PID=75654 | USER:Zeus UID:25
EVENT: DISK | Instructions [fsync, close, commit, open, fsync, read, fsync, write]
EVENT: RAM | Instructions [move, read, store, store, alloc]
EVENT: DISK | Instructions [sync, sync, commit, flush, flush, fsync, rename, unlink]
EVENT: CPU | Instructions [inc, jmp, ret, add, loop, add, inc]
EVENT: CPU | Instructions [mod, add, div]
EVENT: CPU | Instructions [xor, mov, mul, add, xor, ret]
EVENT: CPU | Instructions [mov, test, pop, jump, inc, dec, div, xor]
doors> pfinish OK
FINISHED:
PID=75654 notepad.exe | STATE: OK | USER:Zeus UID:25

```

Figura 24: Prueba manual del límite de la pila de finalizados

Por último, se probó el comportamiento de la pila de procesos finalizados al alcanzar su capacidad máxima. Como la pila tiene un límite de 3 procesos finalizados en memoria, se agregó un cuarto proceso para provocar el desborde. El sistema registró el contenido anterior y dejó en memoria únicamente el nuevo proceso finalizado. Esto permite verificar que el control de capacidad de la pila funciona correctamente y que los procesos descartados ya no se mantienen en memoria.

Conclusión

Se logró implementar el módulo de administración de procesos y eventos para Doors, cumpliendo con el ciclo de vida definido para los procesos: carga, preparación, ejecución, finalización y consulta de estados. El sistema permite cargar usuarios y procesos desde archivos CSV, calcular la prioridad de cada proceso, ejecutar el proceso pendiente con mayor prioridad y finalizarlo con los estados OK, ERROR o TERMINATED.

La solución se organizó separando las responsabilidades principales. Las clases del dominio representan usuarios, procesos y eventos, `ProcessManagerImpl` concentra la lógica de administración. `ProcessConsole` se encarga de la interacción por consola. y `DoorsLogger` centraliza la escritura del archivo de log. Esta separación permitió mantener el código más ordenado y facilitar el control de cada parte del sistema.

Las estructuras de datos utilizadas fueron elegidas según el comportamiento requerido por cada estado del proceso. La cola permite conservar el orden de llegada de los procesos nuevos, el heap permite obtener el proceso pendiente con mayor prioridad, la pila permite manejar los últimos procesos finalizados y los hash permiten acceder de forma directa a usuarios y procesos mediante UID y PID. En conjunto, estas estructuras permiten cumplir con los requerimientos del sistema sin recorrer información innecesaria en las operaciones principales.

En cuanto al análisis de complejidad, el comando `pload` tiene un comportamiento lineal respecto a la cantidad total de datos cargados desde los archivos. El comando `pprepare` depende del recorrido de eventos y de la inserción de procesos en el heap, por lo que su complejidad queda determinada por $O(E + p \log p)$. La ejecución con `pexecute` aprovecha el heap para obtener el proceso prioritario en $O(\log p)$, sumando el costo de recorrer los eventos e instrucciones que se muestran y registran. La finalización con `pfinish` se mantiene en $O(1)$ para la implementación realizada, ya que la pila de finalizados tiene una capacidad máxima constante. Las consultas con `pstatus` pueden llegar a $O(p \log p)$ cuando se deben

mostrar los procesos pendientes ordenados, y en el caso de `pstatus -verbose` se suma además el recorrido de eventos e instrucciones.

A partir de esto, se puede interpretar que el sistema tiene un comportamiento eficiente para las operaciones principales. Las tareas más costosas son la preparación de procesos y algunas consultas de estado, ya que requieren ordenar o recorrer varios procesos. Por el contrario, la ejecución del próximo proceso, la finalización y las búsquedas directas por PID o UID se resuelven con costos bajos gracias al uso del heap y de las estructuras hash.

Se realizaron pruebas con JUnit sobre las clases principales del dominio, sobre `ProcessManagerImpl` y sobre los TADs utilizados por el proyecto. También se realizaron pruebas manuales desde consola para verificar los comandos principales y el comportamiento del log. Los resultados fueron correctos, ya que se pudo cargar la información, preparar procesos, ejecutar procesos, finalizar con los tres estados posibles, consultar el estado del sistema y comprobar el manejo del desborde de la pila de finalizados.